

B+-tree cost example for index on B

★ Given / Assumptions

★ About the relation $R(A,B,C,D,E,F)$

★ B is int (4 Bytes) within $[0;19999]$ uniform distribution

★ 200,000 tuples

★ About the index to be built on attribute B (which is an int)

★ An indirect index alternative II (one data entry per different value of the search key)

★ An index page has 4K

★ Index pages are filled between 50% - 100% (this always holds!!)

★ Let's assume that they are on average 75% full

★ Size of an rid = 10 Bytes

★ The size of a pointer in intermediate pages (i.e., a page identifier): 8 Bytes

★ Data Entries in Leaf pages

★ Number of data entries:

★ distinct values of B = 20000

★ Number of records with same value / number of rids in a data entry

★ $200,000 / 20,000 = 10$ records per val of B

★ Size of data entry

★ $\text{sizeof}(B) + 10 \text{ rids} * \text{sizeof}(\text{rids})$

★ $4 + 10 * 10 = 104$ bytes/data entry

B+tree cost example contd.

☆ The number of leaf pages

☆ Calculation Alternative I:

☆ Number of data entries per leaf page

☆ $4000 * 0.75 / 104 \approx 28.846$ data entries/page on an average.

☆ Number of leaf pages: $20,000 / 29 \approx 690$ leaf pages

☆ Calculation Alternative II:

☆ Number of data entries * size of data entry / occupied space on leaf page

☆ $20,000 * 104 / 3000 = 693$

☆ The height of the tree

☆ Size of an index entry in intermediate node

☆ $4 + 8 = 12$ bytes

☆ Max. number of index entry for intermediate page:

☆ $4000 / 12 = 333$

☆ A single root node cannot hold pointers to all leaf pages. Thus, height > 1

☆ intermediate nodes at least 50% (167 index entries), max 100% (333 index entries)

☆ They have a total of around 690 pointers to leaf nodes

☆ Thus, minimum 3 intermediate nodes right above the leaf pages (to manage 690 leaves).

☆ One node above them to hold pointers to the 3 intermediate pages - root.

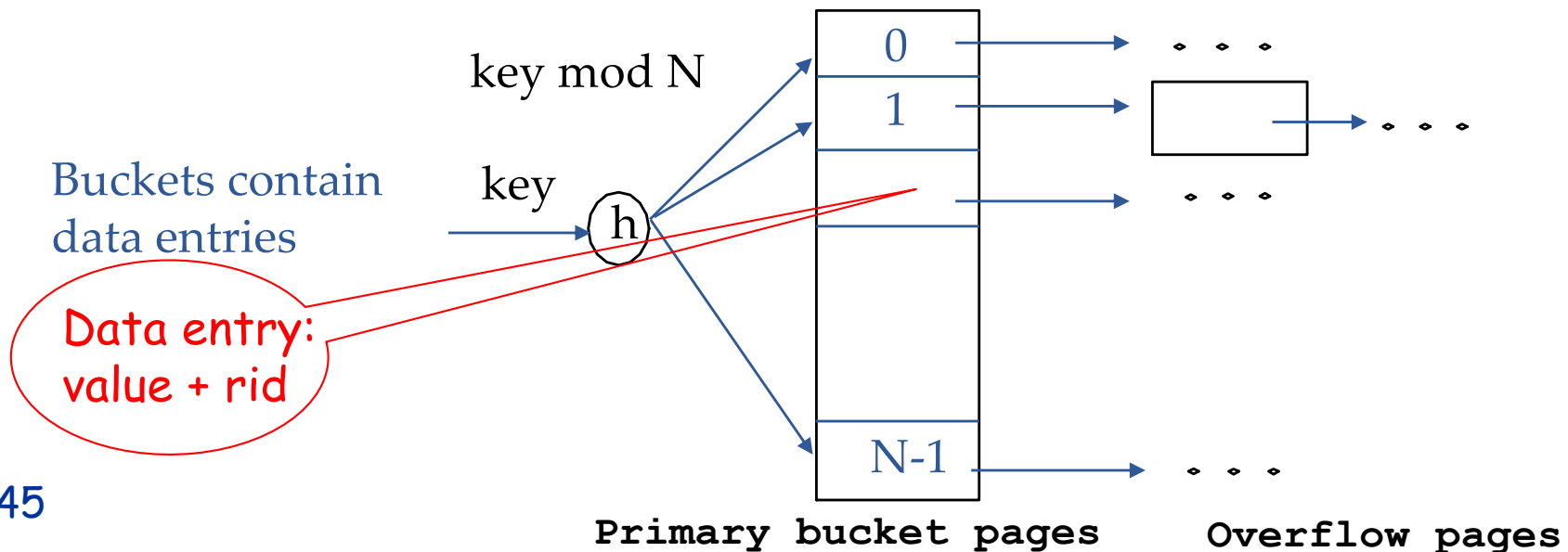
☆ height = 2 (the number of edges from root to a leaf node)

B+ Trees in Practice

- ❑ Typical max. number of index entries in inner nodes: 200
 - ❑ (I.e., an inner node has between 100 and 200 index entries)
 - ☆ Typical fill-factor: 67%.
 - ☆ average fanout = 133
- ❑ Leaf nodes have often less entries since data entries larger (rids)
 - ❑ For simplicity: assume 100
- ❑ Typical capacities (roughly)
 - ☆ Height 1: $133 * 100 = 13,300$ records (just the root and leaves)
 - ☆ Height 2: $133^2 * 100 = 1,768,900$ records
 - ☆ Height 3: $133^3 * 100 = 235,263,700$ records
- ❑ Can often hold top levels in buffer pool:
 - ☆ Level 1 (root) = 1 page = 4 Kbytes
 - ☆ Level 2 = 133 pages = 0.5 Mbyte
 - ☆ Level 3 = 17,689 pages = 70 MBytes

Static Hashing

- ❖ Similar to standard hashing but with pages as the unit of storage (compare with array-based main memory implementation)
- ❖ Decide on a number N of buckets at index creation
- ❖ Allocate one primary page per bucket
- ❖ Overflow pages for individual buckets are created later as needed
- ❖ Buckets contain data entries (same as leaf pages in tree)
- ❖ let k be the search key of the index, h a hash function
 - ❖ E.g. $h = \text{mod}$: $k \bmod N$ = bucket to which data entry with key k belongs.



contd

- ❖ Buckets contain *data entries*.
- ❖ Hash **h** works on *search key* field of record *r*.
Must distribute values over range 0 ... N-1.
 - **h**(key) = (a * key + b) usually works well.
 - a and b are constants; lots known about how to tune **h**.
- ❖ Long overflow chains can develop and degrade performance.
 - ❖ Several optimizations developed such to handle scale dynamically (e.g., extensible and linear hashing)

What is hash index good for

- Point Query
 - `SELECT * FROM Students WHERE sid = 100`
- Equality Query
 - `SELECT * FROM Students WHERE starty = 2015`
- Range Search
 - `SELECT * FROM Students`
`WHERE starty > 2019 and starty <= 2022`
- Costs?
- Initialization costs?